

Dual-Insight Trader: Integrating FinRL Stock Selection with FinGPT Sentiment Analysis for Quantitative Trading

Tiantian Hang(Mentored by Bruce Yang)

December 12, 2025

Abstract

This report conducts a Dual-Insight Trader, an integrated quantitative trading system that combines FinRL-based stock selection with FinGPT-powered sentiment analysis. The system leverages machine learning models for fundamental stock selection and large language models for sentiment-driven signal generation. Through comprehensive backtesting on S&P 500 stocks from December 2024 to December 2025, the system achieved an annualized return of 17.11% with a Sharpe ratio of 0.7080. The integration demonstrates the complementary nature of technical analysis and sentiment analysis in portfolio construction, with sentiment signals serving as filters to enhance risk-adjusted returns.

Keywords: Quantitative Trading, FinRL, FinGPT, Sentiment Analysis, Portfolio Optimization, Machine Learning

1 Introduction

Recent advances in machine learning and natural language processing have transformed how quantitative traders approach market analysis and decision-making. Traditional quantitative strategies rely primarily on technical indicators and fundamental analysis, while large language models (LLMs) enable the extraction of valuable insights from unstructured financial news and textual data.

This project addresses the challenge of combining two distinct approaches:

- **FinRL-based stock selection:** Uses machine learning models (LSTM, Random Forest, SVM, etc.) to predict stock returns based on fundamental and technical features
- **FinGPT sentiment analysis:** Leverages fine-tuned LLMs to generate trading signals from financial news and company information

This research aims to address three primary objectives. First, we seek to establish a novel integration framework that combines FinRL’s machine learning-based stock selection capabilities with FinGPT’s sentiment signal generation, creating a unified approach to market analysis. Second, we endeavor to develop a sophisticated portfolio management system that optimally synthesizes both technical indicators and sentiment-derived signals, leveraging the complementary strengths of each information source. Finally, we conduct comprehensive backtesting to evaluate the integrated system’s performance, which focus on analyzing how each component contributes to the overall portfolio returns and risk characteristics.

This report is organized as follows: Section 2 reviews the relevant methodology. Section 3 describes the system architecture and implementation. Section 4 presents the experimental setup and results. Section 5 discusses the findings and limitations. Section 6 concludes with future work.

2 Methodology

2.1 FinRL for Stock Selection

The stock selection module follows the dynamic stock recommendation framework of Yang et al. [1], adapted to our FinRL-based trading system. Rather than selecting stocks directly from prices, we first use supervised learning models to predict forward returns from firm-level fundamentals on a sector-by-sector basis. The resulting predicted returns are then used to construct a dynamically updated candidate universe for the technical agent and the mean-variance portfolio optimizer.

2.1.1 Fundamental Panel and Target Definition

Let $X_{t,i} \in \mathbb{R}^p$ denote the vector of fundamental ratios for stock i at reporting date t , extracted from the processed fundamental table `final_ratios.csv`. The panel includes standard accounting-based indicators (e.g., profitability, leverage, valuation, and growth ratios) along with sector identifiers and metadata. In the implementation, non-feature columns such as identifiers and date fields (e.g., `gvkey`, `tic`, `datadate`, `fyearq`, `gsector`) are explicitly excluded, and only numeric columns without missing values are retained as features.

The prediction target is the forward quarterly return

$$r_{t+1,i} = \log \frac{S_{t+1,i}}{S_{t,i}},$$

stored in the column `y_return`, where $S_{t,i}$ denotes the adjusted price at quarter t . Following [1], we model the conditional expectation of $r_{t+1,i}$ as

$$r_{t+1,i} = g_\theta(X_{t,i}) + \varepsilon_{t,i},$$

where g_θ is a regression function with parameters θ and $\varepsilon_{t,i}$ is a zero-mean noise term.

To account for structural heterogeneity across industries, we fit separate models for each sector s (e.g., GICS sectors 10, 15, ..., 60). For sector s , we construct a sector-specific panel $\{X_{t,i}^{(s)}, r_{t+1,i}^{(s)}\}$ over all firms i with sector label s and all available quarter-end dates t .

2.1.2 Rolling-Window Training and Dynamic Model Selection

The relationship between fundamentals and future returns is potentially time-varying. To capture this, we employ a rolling-window procedure that mirrors the code implementation in `fundamental_run_model.py` and `ml_model.py`. For each sector, the quarterly dates are sorted to yield a sequence $\{\tau_1, \tau_2, \dots, \tau_T\}$, and a fixed index i_0 (e.g., $i_0 = 20$) marks the first date at which trading decisions are made.

For each current index $i \in \{i_0, \dots, T-1\}$, we define three disjoint time segments:

- **Training window:** a multi-year history ending before the validation window, from τ_ℓ to τ_{i-w} , where w is the validation length (e.g., $w = 4$ quarters), and ℓ is chosen such that at most a fixed number of quarters (e.g., 44) are used if the history is very long.
- **Validation window:** the subsequent w quarters, from τ_{i-w+1} to τ_i , used to evaluate competing models.
- **Trade date:** the next quarter τ_{i+1} , for which we generate return forecasts to support portfolio formation.

Within each sector and at each rolling step, we consider a small model zoo

$$\{g_\theta^{(m)} : m \in \{\text{RF}, \text{XGB}, \text{GBM}\}\},$$

consisting of a random forest regressor, an XGBoost regressor, and a LightGBM regressor, as implemented in `ml_model.py`. For each model m we:

1. Fit $g_\theta^{(m)}$ on the training window using the sector-specific features $X_{t,i}^{(s)}$ and targets $r_{t+1,i}^{(s)}$.

2. Evaluate its out-of-sample performance on the validation window via the mean squared error (MSE)

$$\text{MSE}^{(m)} = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(t,i) \in \mathcal{D}_{\text{val}}} (r_{t+1,i}^{(s)} - g_{\theta}^{(m)}(X_{t,i}^{(s)}))^2,$$

as computed by the `evaluate_model` function.

3. Use the fitted model to predict returns for all stocks in sector s at the trade date τ_{i+1} , obtaining a vector of predicted returns $\hat{r}_{\tau_{i+1},\cdot}^{(m)}$.

We then form a small evaluation table containing, for each model, its validation error and corresponding trade-date predictions. The best model at time τ_{i+1} is selected by minimizing the validation MSE:

$$m^*(\tau_{i+1}) = \arg \min_{m \in \{\text{RF}, \text{XGB}, \text{GBM}\}} \text{MSE}^{(m)}.$$

The corresponding predicted returns

$$\hat{r}_{\tau_{i+1},\cdot}^{\text{best}} = \hat{r}_{\tau_{i+1},\cdot}^{(m^*(\tau_{i+1}))}$$

are stored in a sector-level prediction table `df_predict_best`.

2.1.3 Sector-Level Ranking and Candidate Set Construction

The script `stock_selection.py` aggregates the sector-level predictions into a global stock selection list. For each sector s and each trade date τ_{i+1} , it reads the corresponding row of `df_predict_best` and obtains the predicted returns $\hat{r}_{\tau_{i+1},i}^{\text{best}}$ across all firms in that sector. These predictions are then converted into a sector-neutral ranking:

1. For sector s at date τ_{i+1} , compute the empirical 75th percentile $q_{0.75}$ of $\hat{r}_{\tau_{i+1},\cdot}^{\text{best}}$.
2. Select the subset of firms with $\hat{r}_{\tau_{i+1},i}^{\text{best}} \geq q_{0.75}$.
3. Record for each selected stock its identifier (`gvkey`), predicted return, and trade date.

Repeating this process over all sectors and all trade dates yields the final stock selection file, which contains the top-quartile predicted performers in each sector on each rebalancing date. This sector-wise top-25% rule is a discrete analogue of the “top-20% within industry” rule used in [1], and serves to control for sector biases while focusing on stocks with the most favorable fundamental outlook.

2.1.4 Integration with FinRL Environment

In the Dual-Insight Trader, the fundamental stock selection module defines the investable set for the downstream trading components. On each rebalancing date, FinRL and the mean–variance portfolio optimizer only consider stocks that appear in the final selected stocks for that date; all other names are effectively assigned zero weight. The technical agent then operates on this filtered set using price-based state variables, while the portfolio manager further integrates FinGPT sentiment signals as described in the following subsections.

Thus, the stock selection pipeline transforms high-dimensional fundamental information into a rolling, sector-neutral list of candidate stocks via supervised learning and dynamic model selection. FinRL builds on this selected stock set to learn trading policies that translate predicted cross-sectional return differentials into realized portfolio performance.

2.2 FinGPT for Sentiment Analysis

FinGPT is a financial domain-specific large language model that generates structured sentiment and return forecasts from real-world news and basic financials. In this project, we adopt the FinGPT-Forecaster checkpoint

released for the Dow Jones 30 universe, which is obtained by LoRA fine-tuning a Llama-3.1 family model on instruction-style prompts constructed from Dow 30 stocks between May 2023 and May 2024.¹ In our code, the sentiment agent loads this LoRA adapter on top of the base model and focuses on efficient inference and signal extraction rather than re-training the model from scratch.

2.2.1 Data and Prompt Design

Each FinGPT training sample is built from a single stock and a fixed lookback window of recent market information. The prompt contains:

- a short company introduction and basic metadata (sector, listing exchange, market capitalization, country);
- a natural-language description of the recent price movement over the past few weeks;
- several recent news headlines and summaries for the company;
- a small table of basic financial ratios (e.g., valuation, profitability, leverage).

A system-level instruction asks the model to behave as a professional equity analyst, first listing key *positive developments* and *potential concerns*, and then providing a short *prediction and analysis* for the stock’s price movement over the next week, including a directional forecast and a rough percentage range. The full system prompt and user template used for training are reported in Appendix A.1.

During inference, our **SentimentAgent** re-uses the same structure: it collects recent news via Finnhub, queries a prompt builder to assemble the company description, price path, news snippets, and basic financials into the training-style template, and then feeds the resulting text to the fine-tuned FinGPT model.

2.2.2 LoRA Fine-Tuning Procedure (Summary)

The FinGPT-Forecaster checkpoint is obtained using the standard LoRA fine-tuning pipeline described in the course assignment.² The main steps are:

1. **Base model and tokenizer:** download the pretrained Llama-3.1 model from HuggingFace together with its tokenizer, enabling mixed-precision and 8-bit loading to fit into a single GPU.
2. **Dataset preparation:** load the Dow 30 prompt dataset generated by the data pipeline (Section 3.1 of the assignment), which already contains train/test splits of (**prompt**, **answer**) pairs.
3. **LoRA configuration:** wrap the base model with a LoRA adapter using the configuration specified in the official script (rank, scaling factor, dropout, and target modules are the same as in our implementation code). Only the LoRA parameters are trainable; all original model weights remain frozen.
4. **Training:** run `train_lora.py` with the provided hyperparameters (sequence length, batch size, gradient accumulation steps, learning rate, number of epochs, warmup ratio, and scheduler) using DeepSpeed; training progress is monitored via W&B.³

Listing 1: FinGPT-Llama3.1 LoRA Configuration

```

1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 from peft import LoraConfig, get_peft_model
3
4 # LoRA Configuration
5 lora_config = LoraConfig(
6     r=16,                                # LoRA rank

```

¹Training details follow the official FinGPT-Forecaster assignment handout [?].

²We follow the official `train_lora.py` and `train.sh` configuration; only a brief summary is given here.

³We keep the original training hyperparameters unchanged and therefore omit them here for brevity.

```

7     lora_alpha=32,                # LoRA scaling parameter
8     target_modules=["q_proj", "v_proj"], # Target modules
9     lora_dropout=0.1,            # Dropout rate
10 )
11
12 # Load and apply LoRA
13 model = AutoModelForCausalLM.from_pretrained(
14     "meta-llama/Llama-3.1",
15     torch_dtype=torch.bfloat16
16 )
17 model = get_peft_model(model, lora_config)

```

This setup injects low-rank adapters into the query and value projection layers of each attention block. Only the adapter parameters are updated during training, which significantly reduces the number of trainable parameters while preserving the expressiveness and stability of the base model.

2.2.3 Inference and Signal Extraction

On each trading date, the sentiment agent operates as follows:

1. Retrieve recent news and basic financials for stock i and date t via the data fetcher.
2. Construct a training-style prompt with company introduction, recent price movement, news headlines and summaries, and basic ratios.
3. Query the fine-tuned FinGPT model to generate a structured response containing:
 - a bullet list of **positive developments**;
 - a bullet list of **potential concerns**;
 - a **prediction & analysis** section with a qualitative price forecast and a rough percentage range for next week.
4. Parse the generated text to extract:
 - a predicted weekly return (mapped from the percentage range in the prediction block);
 - a sentiment confidence score (based on the magnitude of the predicted return and the balance between positive and negative factors);
 - auxiliary statistics such as the number of positive and negative bullet points.

These numerical outputs are then transformed into trading signals: strong positive (negative) predicted returns correspond to buy (sell) recommendations with high confidence, while near-zero predictions lead to a neutral signal. In the Dual-Insight Trader, we primarily use the predicted return as a sentiment-based filter in the portfolio construction stage (e.g., excluding stocks with predicted returns below -2.5%) and as an additional feature when combining sentiment with technical signals.

2.3 Portfolio Optimization

The final portfolio is constructed by combining a sentiment-based pre-trade filter with a mean-variance optimizer. Let $\mathcal{U}_t$ denote the set of stocks available at rebalancing time t (after fundamental stock selection), and let $\hat{r}_{t+1,i}^{\text{sent}}$ be the one-period predicted return from FinGPT for stock $i \in \mathcal{U}_t$.

Sentiment-based pre-trade filter. As implemented in `PortfolioManager`, we first apply a hard filter to exclude names with strongly negative sentiment forecasts. Specifically, we define a sentiment threshold

$$\tau = -0.025,$$

corresponding to a -2.5% predicted return over the next period. The sentiment-valid universe at time t is

$$\mathcal{V}_t = \{i \in \mathcal{U}_t : \hat{r}_{t+1,i}^{\text{sent}} \geq \tau\}.$$

If $\mathcal{V}_t$ is non-empty, only stocks in $\mathcal{V}_t$ are considered in the subsequent optimization; if all stocks are filtered out ($\mathcal{V}_t = \emptyset$), the portfolio manager records this event and refrains from constructing a new sentiment-filtered allocation for that date.

Mean–variance optimization with technical signals. Conditional on the sentiment-valid universe $\mathcal{V}_t$, we adopt a Markowitz mean–variance optimization based on technical signals. The technical agent (Section 2.1) produces, for each date t , a preliminary set of mean–variance weights

$$w_t^{\text{tech}} = (w_{t,i}^{\text{tech}})_{i \in \mathcal{U}_t}, \quad w_{t,i}^{\text{tech}} \geq 0, \quad \sum_{i \in \mathcal{U}_t} w_{t,i}^{\text{tech}} = 1,$$

obtained by solving a classical portfolio problem of the form

$$\max_w w^\top \hat{\mu}_t - \frac{\lambda}{2} w^\top \Sigma_t w,$$

subject to long-only and budget constraints. Here $\hat{\mu}_t$ denotes expected returns inferred from technical features, Σ_t is a covariance estimate based on recent return history, and $\lambda > 0$ controls risk aversion.

The portfolio manager then projects these technical weights onto the sentiment-valid universe by zeroing out all positions that fail the sentiment filter and renormalizing the remaining weights:

$$w_{t,i}^* = \begin{cases} \frac{w_{t,i}^{\text{tech}}}{\sum_{j \in \mathcal{V}_t} w_{t,j}^{\text{tech}}}, & i \in \mathcal{V}_t, \\ 0, & i \notin \mathcal{V}_t. \end{cases}$$

This procedure matches the implementation in `_apply_technical_weights`, where the technical mean–variance solution is used as the primary source of portfolio weights, and sentiment plays a purely filtering role rather than directly entering the objective function.

Rebalancing schedule and outputs. Consistent with the fundamental data frequency, the optimization is performed on a quarterly grid: at each quarter-end t , the sentiment filter and mean–variance optimizer jointly determine the new weight vector w_t^* , which is held until the next rebalancing date. For each decision, the portfolio manager records the set of filtered-out stocks, the final weight vector, and auxiliary diagnostics such as the Sharpe ratio reported by the technical agent and an internal confidence score. These records are subsequently used in the empirical analysis to quantify the impact of sentiment filtering on portfolio composition and risk–return characteristics.

3 System Implementation and Reproducibility

3.1 Codebase Overview

The implementation follows the framework shown in Figure 1. The project is organized into three main layers:

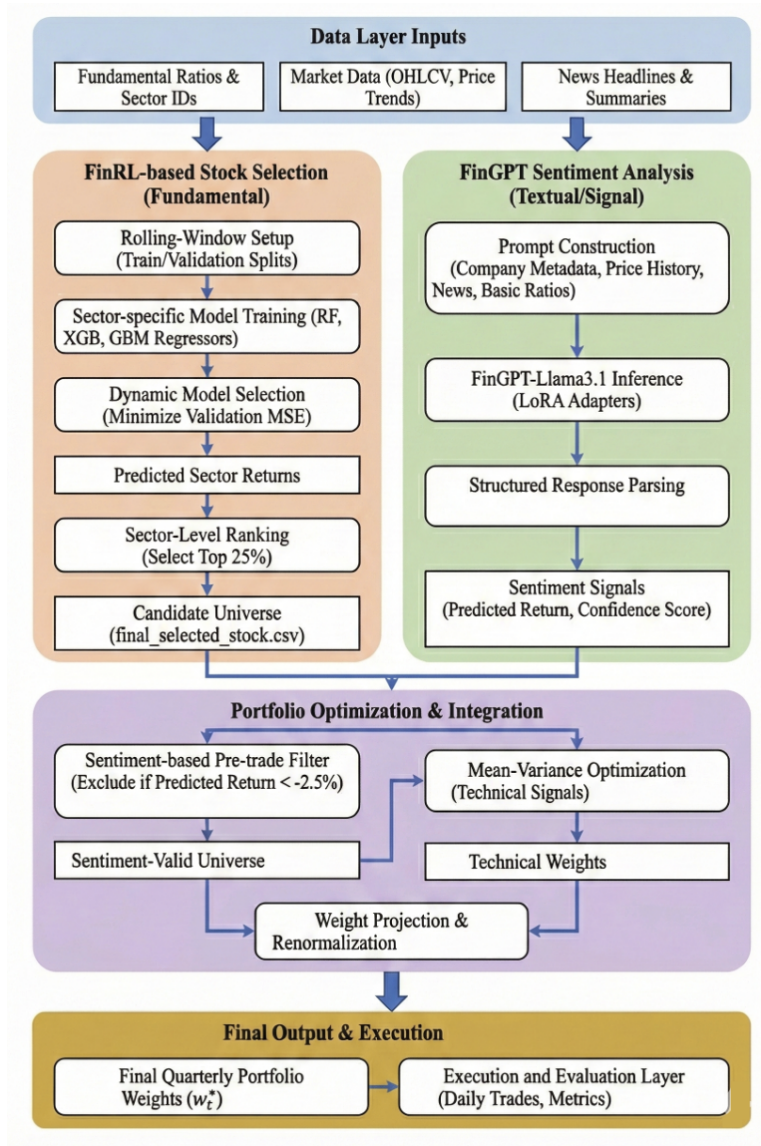


Figure 1: Overall framework

- **Fundamental stock selection:** implemented in `fundamental_run_model.py` and `ml_model.py`. These scripts:
 1. load the processed fundamental panel `final_ratios.csv` and construct sector-specific datasets;
 2. perform rolling train/validation splits on quarterly dates;
 3. train random forest, XGBoost, and LightGBM regressors within each sector;
 4. select the model with the lowest validation MSE and generate sector-level return predictions;
 5. aggregate the top-25% predicted performers into `final_selected_stock.csv`, which defines the candidate universe.
- **FinGPT sentiment analysis:** implemented in `sentiment_agent.py`. The sentiment agent:
 1. fetches recent news, price history, and basic financials for each stock;
 2. constructs prompts using the template described in Section 2.2 and Appendix A.1;
 3. loads the released FinGPT-Forecaster LoRA checkpoint on top of the Llama-3.1 base model;
 4. runs inference and parses the structured response into a predicted weekly return and a sentiment confidence score.

- **Portfolio optimization and backtesting:** implemented `portfolio_manager.py` and `stock_selection.py`. These modules:

1. intersect the fundamental candidate universe with the set of stocks covered by the sentiment agent;
2. apply the sentiment-based pre-trade filter (exclude names with predicted return below -2.5%) to obtain the sentiment-valid universe;
3. obtain technical mean–variance weights from the FinRL agent and project them onto the sentiment-valid set via renormalization;
4. simulate quarterly rebalancing and record portfolio value, weights, and diagnostics for evaluation.

3.2 Computing Environment and Dependencies

All experiments are conducted on a Linux machine equipped with a single NVIDIA A100 GPU and at least 40 GB of GPU memory for running the FinGPT model. The implementation relies on standard Python libraries:

- `pandas`, `numpy` for data manipulation;
- `scikit-learn`, `xgboost`, `lightgbm` for fundamental regressors;
- `transformers` and `peft` for loading the Llama-3.1 base model and the FinGPT LoRA adapter;
- `matplotlib` for plotting portfolio value and performance metrics.

A single configuration file (e.g., `config.yaml`) stores the main hyperparameters such as training window length, validation window length, sector list, and sentiment threshold. The random seeds for both the machine learning models and the backtesting environment are fixed to ensure that reported results can be reproduced.

4 Experimental Results and Analysis

4.1 Experimental Setup

We evaluate the Dual-Insight Trader, and the backtest is conducted on a daily trading calendar with quarterly rebalancing dates aligned with the release of fundamental data. On each rebalancing date, the system:

1. constructs a sector-neutral candidate universe using the fundamental stock selection module;
2. generates FinGPT-based sentiment forecasts and applies the sentiment filter with a threshold of -2.5% predicted weekly return;
3. solves a mean–variance problem using technical signals and projects the resulting weights onto the sentiment-valid universe.

The resulting portfolio weights are held until the next quarter-end. Portfolio value is updated daily using close-to-close returns, with a proportional transaction cost applied to all trades.

As a market benchmark, we use the total return of the S&P 500 index (denoted SPX500), computed over the same sample period. This allows us to compare the Dual-Insight Trader not only in absolute terms, but also relative to a broad, diversified equity index representing the U.S. market portfolio.

We use standard risk–return metrics, total return, annualized return, annualized volatility, Sharpe ratio (using the risk-free rate assumed in the assignment), and Calmar ratio (annualized return divided by maximum drawdown), to evaluate performance.

Table 1: Dual-Insight Trader vs. S&P 500 (SPX500) Performance Comparison

Metric	Dual-Insight Trader	SPX500	Difference
Total Return	16.07%	13.56%	2.51%
Annualized Return	17.11%	13.56%	3.55%
Annualized Volatility	21.34%	18.96%	2.38%
Sharpe Ratio	0.7080	0.7152	−0.0072
Calmar Ratio	0.8282	0.7175	0.1107

4.2 Portfolio Performance Relative to S&P 500

Table 1 reports the key performance statistics for the Dual-Insight Trader and the SPX500 benchmark over the evaluation period.

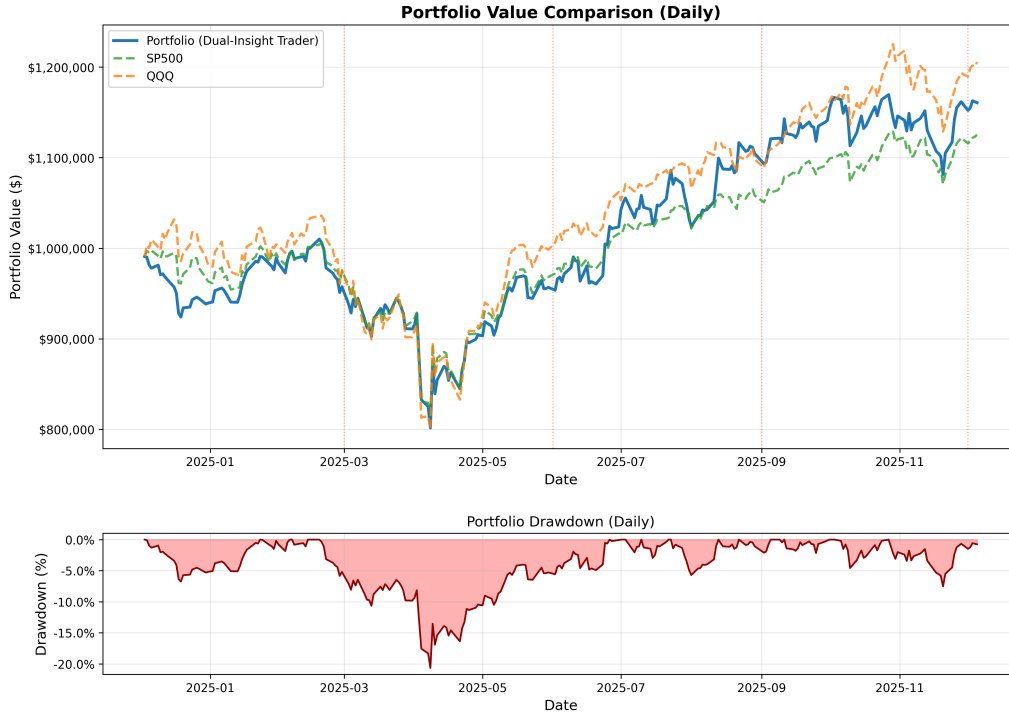


Figure 2: Daily portfolio value of the Dual-Insight Trader compared with SP500 and QQQ from 2024 to 2025 (top), and the corresponding drawdown profile of the Dual-Insight portfolio (bottom). Vertical dashed lines indicate quarterly rebalancing dates.

The Dual-Insight Trader achieves a higher total return (16.07%) and annualized return (17.11%) than the SPX500 benchmark (13.56% for both measures), corresponding to an outperformance of 2.51 percentage points in total return and 3.55 percentage points in annualized terms. This indicates that, over the sample period, the sentiment-augmented strategy generates incremental alpha relative to a broad passive index.

The higher return comes at the cost of somewhat higher risk: annualized volatility is 21.34% for the Dual-Insight Trader versus 18.96% for SPX500, an increase of 2.38 percentage points. As a result, the Sharpe ratios of the two strategies are very similar (0.7080 vs. 0.7152), suggesting that the Dual-Insight Trader delivers excess return primarily by taking on moderately more volatility, while maintaining a comparable level of risk-adjusted performance.

However, the Calmar ratio is noticeably higher for the Dual-Insight Trader (0.8282 vs. 0.7175). Since the Calmar ratio jointly reflects return and maximum drawdown, this improvement indicates that, relative to its peak-to-trough losses, the Dual-Insight strategy allocates risk more efficiently than the index benchmark. In particular, the sentiment filter helps the portfolio sidestep part of the downside associated with stocks that exhibit persistently negative narratives, which mitigates drawdowns despite the higher day-to-day volatility.

Overall, these results show that the Dual-Insight Trader behaves as an active, moderately more volatile strategy that: (i) outperforms the S&P 500 in terms of realized returns; (ii) achieves risk-adjusted performance roughly in line with the benchmark as measured by the Sharpe ratio; and (iii) exhibits superior drawdown-adjusted performance as captured by the Calmar ratio. This suggests that combining fundamental stock selection, FinGPT-based sentiment filtering, and mean–variance optimization can produce a portfolio that is competitive with, and in several dimensions stronger than, a passive exposure to the broad U.S. equity market.

5 Discussion and Conclusion

This paper proposed a Dual-Insight Trader, an integrated system that combines FinRL-driven stock selection with FinGPT-based sentiment filtering and embeds the resulting signals into a mean–variance portfolio construction framework.

The Dual-Insight Trader achieves an annualized return of 17.11% with a Sharpe ratio of 0.7080, compared with 13.56% annualized return and a Sharpe ratio of 0.7152 for the S&P 500 (SPX500) benchmark over the same sample period. This strategy delivers higher realized returns than a broad market index at a comparable level of risk-adjusted performance. In addition, the Dual-Insight Trader obtains a higher Calmar ratio than SPX500, indicating that, relative to its peak-to-trough losses, the strategy allocates risk more efficiently and manages drawdowns more effectively than the passive benchmark. The increase in total and annualized returns compared to SPX500 suggests that FinGPT can filter out information from news and fundamentals that is not captured by index-level exposures alone and that these additional signals translate into economically meaningful gains.

This paper shows that combining a sentiment-based pre-filter with mean–variance optimization yields a transparent and implementable framework for incorporating LLM signals into portfolio construction: the sentiment layer reduces the investable sets to assets with favorable news signals, and classical optimization is then used to determine position sizes. Taken together, these findings provide evidence that LLM-based sentiment can be a useful building block for active equity strategies that seek to outperform broad market benchmarks while maintaining disciplined risk control.

A Appendix

A.1 FinGPT Training Prompt Template

The FinGPT forecaster is trained with a two-part prompt consisting of a system instruction and a stock-specific user template.

System Prompt (Sketch). The system prompt instructs the model to act as a seasoned stock market analyst. It asks the model to:

- read recent news and basic financials for a given company;
- list 2–4 key *positive developments* and 2–4 key *potential concerns*, in bullet form;
- provide a short *prediction & analysis* paragraph that summarizes the near-term outlook and predicts the stock price movement for the upcoming week, including a rough percentage change.

User Prompt Template. For each training sample, the user part of the prompt follows a fixed structure:

- **Company introduction:** a short paragraph describing the firm, its sector, listing exchange, market capitalization, and primary country of operation.
- **Recent price movement:** a sentence describing how the stock price has moved from a start date to an end date (increase/decrease and magnitude).

- **News block:** several pairs of [Headline] and [Summary] lines covering company-related news during the lookback window.
- **Basic financials:** a list of key ratios presented as [Basic Financials]: metric : value.
- **Instruction:** a closing paragraph that asks the model to (i) extract 2–4 positive developments and 2–4 potential concerns based mainly on the news, and (ii) predict the stock price movement for next week with a brief justification.

In our implementation, the **SentimentAgent** reconstructs prompts at inference time using exactly this structure, ensuring that the deployed model receives inputs that are consistent with its training distribution.

B Hyperparameters

Table 2: Key System Hyperparameters

Parameter	Value	Description
Sentiment Threshold	-0.025	Minimum predicted return for inclusion
Top Stock Percentage	25%	Percentage of stocks to select
Rebalancing Frequency	Quarterly	Portfolio rebalancing period
Transaction Cost	0.1%	Cost per trade
Risk Aversion (λ)	1.0	Mean-variance optimization parameter

C Core Source Code

C.1 Portfolio Manager

Portfolio Manager - The Portfolio Manager is the final decision-maker that combines signals from Stock Selection, Sentiment Agent, and Technical Agent to generate optimal portfolio weights. It uses a two-stage approach: sentiment filtering followed by mean-variance optimization.

Listing 2: Portfolio Manager (portfolio_manager.py)

```

1  """
2  Portfolio Manager - Uses Mean-Variance Optimization with Sentiment Filtering
3  New Logic:
4  1. Filter stocks: Exclude stocks with sentiment predicted_return < -0.025
5  2. Use Technical weights: Apply mean-variance optimized weights from Technical Agent
6  3. No weight fusion: Sentiment is used as filter, not for weight calculation
7  """
8
9  import numpy as np
10 import pandas as pd
11 from typing import Dict, List, Optional
12
13
14 class PortfolioManager:
15     """
16     Portfolio manager using mean-variance optimization with sentiment filtering
17
18     New Strategy:
19     - Sentiment acts as a filter: exclude stocks with predicted_return < -0.025
20     - Technical provides optimized weights using mean-variance optimization
21     - No weight fusion between sentiment and technical
22     """
23
24     def __init__(self, sentiment_threshold=-0.025, use_optimization=True):

```

```

25     """
26     Initialize portfolio manager
27
28     Args:
29         sentiment_threshold: Exclude stocks with sentiment predicted_return below this
30                             threshold
31         use_optimization: Whether to use mean-variance optimized weights from technical
32                             """
33     self.sentiment_threshold = sentiment_threshold
34     self.use_optimization = use_optimization
35
36 def make_decision(self, sentiment_result, technical_result, market_state=None):
37     """
38     Generate portfolio weights using new logic:
39     1. Filter stocks based on sentiment predicted_return
40     2. Apply technical optimized weights to remaining stocks
41
42     Args:
43         sentiment_result: dict or DataFrame with sentiment predictions
44             - If dict: {'gvkey': {'predicted_return': float, ...}, ...}
45             - If DataFrame: columns=['gvkey', 'predicted_return', ...]
46         technical_result: dict from TechnicalAgentOptimized
47             - Must contain 'weights': {gvkey: weight}
48             - May contain 'expected_returns', 'sharpe_ratio', 'confidence'
49         market_state: optional market condition dict (not used in new logic)
50
51     Returns:
52         dict: {
53             'weights': {gvkey: weight}, # Final portfolio weights
54             'filtered_stocks': [gvkey], # Stocks excluded by sentiment filter
55             'confidence': float,
56             'sharpe_ratio': float, # If available from technical
57             'reasoning': str,
58             'method': str
59         }
60     """
61     # Convert sentiment_result to dict format if needed
62     sentiment_dict = self._normalize_sentiment_result(sentiment_result)
63
64     # Get technical weights
65     if not isinstance(technical_result, dict) or 'weights' not in technical_result:
66         raise ValueError(
67             "technical_result must be a dict with 'weights' key. "
68             "Use TechnicalAgentOptimized.analyze_portfolio() to generate weights."
69         )
70
71     technical_weights = technical_result['weights']
72
73     # Step 1: Filter stocks based on sentiment predicted_return
74     filtered_stocks, valid_stocks = self._filter_by_sentiment(
75         sentiment_dict,
76         technical_weights
77     )
78
79     if len(valid_stocks) == 0:
80         # No valid stocks after filtering
81         return {
82             'weights': {},
83             'filtered_stocks': list(filtered_stocks),
84             'confidence': 0.0,
85             'sharpe_ratio': None,

```

```

85         'reasoning': f"All stocks filtered out (sentiment predicted_return <
86             {self.sentiment_threshold})",
87         'method': 'sentiment_filtered'
88     }
89
90     # Step 2: Apply technical weights to valid stocks
91     final_weights = self._apply_technical_weights(technical_weights, valid_stocks)
92
93     # Step 3: Calculate confidence and metrics
94     confidence = technical_result.get('confidence', 0.5)
95     sharpe_ratio = technical_result.get('sharpe_ratio', None)
96
97     reasoning = (
98         f"Filtered {len(filtered_stocks)} stocks with sentiment predicted_return <
99             {self.sentiment_threshold}. "
100         f"Applied mean-variance optimized weights to {len(valid_stocks)} stocks. "
101     )
102     if sharpe_ratio is not None:
103         reasoning += f"Portfolio Sharpe Ratio: {sharpe_ratio:.3f}"
104
105     return {
106         'weights': final_weights,
107         'filtered_stocks': list(filtered_stocks),
108         'confidence': confidence,
109         'sharpe_ratio': sharpe_ratio,
110         'expected_return': technical_result.get('expected_return', None),
111         'volatility': technical_result.get('volatility', None),
112         'reasoning': reasoning,
113         'method': 'mean_variance_optimized_with_sentiment_filter'
114     }
115
116 def _normalize_sentiment_result(self, sentiment_result):
117     """
118     Normalize sentiment result to dict format: {gvkey: {'predicted_return': float, ...}}
119
120     Args:
121     sentiment_result: Can be:
122         - dict: {gvkey: {'predicted_return': float, ...}}
123         - DataFrame: with columns ['gvkey', 'predicted_return', ...]
124         - dict: {'stocks': [{gvkey: ..., predicted_return: ...}, ...]}
125
126     Returns:
127     dict: {gvkey: {'predicted_return': float, ...}}
128     """
129     if isinstance(sentiment_result, pd.DataFrame):
130         # DataFrame format
131         if 'gvkey' not in sentiment_result.columns:
132             raise ValueError("sentiment_result DataFrame must have 'gvkey' column")
133         if 'predicted_return' not in sentiment_result.columns:
134             raise ValueError("sentiment_result DataFrame must have 'predicted_return' column")
135
136         result_dict = {}
137         for _, row in sentiment_result.iterrows():
138             gvkey = row['gvkey']
139             result_dict[gvkey] = {
140                 'predicted_return': float(row['predicted_return']),
141                 **{k: v for k, v in row.items() if k not in ['gvkey', 'predicted_return']}
142             }
143         return result_dict

```

```

143 elif isinstance(sentiment_result, dict):
144     # Check if it's already in the right format
145     if 'stocks' in sentiment_result:
146         # Format: {'stocks': [{gvkey: ..., predicted_return: ...}, ...]}
147         result_dict = {}
148         for stock_data in sentiment_result['stocks']:
149             if 'gvkey' in stock_data:
150                 gvkey = stock_data['gvkey']
151                 result_dict[gvkey] = {
152                     'predicted_return': float(stock_data.get('predicted_return',
153                                                             0.0)),
154                     **{k: v for k, v in stock_data.items() if k not in ['gvkey',
155                                                             'predicted_return']}
156                 }
157             return result_dict
158         else:
159             # Format: {gvkey: {'predicted_return': float, ...}}
160             # Verify structure
161             for gvkey, data in sentiment_result.items():
162                 if not isinstance(data, dict):
163                     raise ValueError(
164                         f"sentence_result[{gvkey}] must be a dict with
165                         'predicted_return' key"
166                     )
167                 if 'predicted_return' not in data:
168                     raise ValueError(
169                         f"sentence_result[{gvkey}] must have 'predicted_return' key"
170                     )
171             return sentiment_result
172         else:
173             raise TypeError(
174                 f"sentence_result must be dict or DataFrame, got {type(sentiment_result)}"
175             )
176
177 def _filter_by_sentiment(self, sentiment_dict: Dict, technical_weights: Dict) -> tuple:
178     """
179     Filter stocks based on sentiment predicted_return threshold
180
181     Args:
182         sentiment_dict: {gvkey: {'predicted_return': float, ...}}
183         technical_weights: {gvkey: weight}
184
185     Returns:
186         tuple: (filtered_stocks_set, valid_stocks_set)
187     """
188     filtered_stocks = set()
189     valid_stocks = set()
190
191     # Get all stocks that have both sentiment and technical data
192     all_stocks = set(sentiment_dict.keys()) & set(technical_weights.keys())
193
194     for gvkey in all_stocks:
195         predicted_return = sentiment_dict[gvkey].get('predicted_return', 0.0)
196
197         if predicted_return < self.sentiment_threshold:
198             # Filter out: predicted_return < threshold
199             filtered_stocks.add(gvkey)
200         else:
201             # Keep: predicted_return >= threshold
202             valid_stocks.add(gvkey)

```

```

201         return filtered_stocks, valid_stocks
202
203     def _apply_technical_weights(self, technical_weights: Dict, valid_stocks: set) -> Dict:
204         """
205         Apply technical weights to valid stocks and renormalize
206
207         Args:
208             technical_weights: {gvkey: weight} from TechnicalAgentOptimized
209             valid_stocks: set of gvkeys that passed sentiment filter
210
211         Returns:
212             dict: {gvkey: normalized_weight} for valid stocks only
213         """
214         # Extract weights for valid stocks only
215         filtered_weights = {
216             gvkey: technical_weights.get(gvkey, 0.0)
217             for gvkey in valid_stocks
218         }
219
220         # Remove zero weights
221         filtered_weights = {k: v for k, v in filtered_weights.items() if v > 0}
222
223         if len(filtered_weights) == 0:
224             return {}
225
226         # Renormalize to sum to 1
227         total_weight = sum(filtered_weights.values())
228         if total_weight > 0:
229             normalized_weights = {
230                 gvkey: weight / total_weight
231                 for gvkey, weight in filtered_weights.items()
232             }
233         else:
234             # If all weights are zero, use equal weights
235             equal_weight = 1.0 / len(valid_stocks)
236             normalized_weights = {
237                 gvkey: equal_weight
238                 for gvkey in valid_stocks
239             }
240
241         return normalized_weights
242
243     def get_portfolio_weights(self, sentiment_result, technical_result, market_state=None):
244         """
245         Alias for make_decision for clarity
246
247         Returns portfolio weights dictionary: {gvkey: weight}
248         """
249         result = self.make_decision(sentiment_result, technical_result, market_state)
250         return result['weights']
251
252     def get_filtered_stocks(self, sentiment_result, technical_result, market_state=None):
253         """
254         Get list of stocks filtered out by sentiment threshold
255
256         Returns list of gvkeys that were excluded
257         """
258         result = self.make_decision(sentiment_result, technical_result, market_state)
259         return result['filtered_stocks']

```

C.2 Stock Selection

Stock Selection - The Stock Selection module uses supervised learning models (XGBoost, Random Forest, GBM) to predict forward quarterly returns from firm-level fundamentals on a sector-by-sector basis. It selects the top 25% of stocks based on predicted returns.

Listing 3: Stock Selection: Main Script (stock_selection.py)

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Stock Selection Script
5  Convert Jupyter notebook stock selection functionality to command line script
6  """
7
8  import pandas as pd
9  import time
10 import os
11 import argparse
12 import sys
13 import subprocess
14 from pathlib import Path
15
16
17 def run_stock_selection(data_path, output_path):
18     """
19     Run stock selection model
20
21     Args:
22         data_path (str): Input sector files directory
23         output_path (str): Output directory
24     """
25     # Define sector range
26     sectors = range(10, 65, 5)
27
28     # Set data directory path
29     DATA_DIR = data_path
30     FUNDAMENTAL_FILE = os.path.join(DATA_DIR, "final_ratios.csv")
31
32     # gvkey is unique identifier
33     df_dict = {'gvkey': [], 'predicted_return': [], 'trade_date': []}
34
35     # Run stock selection for all sectors
36     for sector in sectors:
37         sector_file = os.path.join(DATA_DIR, f"sector{sector}.xlsx")
38
39         # Run model training
40         script_dir = os.path.dirname(os.path.abspath(__file__))
41         fundamental_abs = os.path.abspath(FUNDAMENTAL_FILE)
42         sector_abs = os.path.abspath(sector_file)
43
44         cmd = [
45             "python3",
46             "fundamental_run_model.py",
47             "-sector_name", f"sector{sector}",
48             "-tic_column", "gvkey",
49             "-fundamental", fundamental_abs,
50             "-sector", sector_abs
51         ]
52
53         subprocess.run(cmd, cwd=script_dir, capture_output=True, text=True)
```



```

55     # Read prediction results
56     result_file = os.path.join(script_dir, f"results/sector{sector}/df_predict_best.csv")
57     df = pd.read_csv(result_file, index_col=0)
58
59     # Select top 25% stocks (75th percentile)
60     for idx in df.index:
61         predicted_return = df.loc[idx]
62         top_q = predicted_return.quantile(0.75)
63         predicted_return = predicted_return[predicted_return >= top_q]
64         for gvkey in predicted_return.index:
65             df_dict["gvkey"].append(gvkey)
66             df_dict["predicted_return"].append(predicted_return[gvkey])
67             df_dict["trade_date"].append(idx)
68
69     # Save results
70     df_result = pd.DataFrame(df_dict)
71     output_file = os.path.join(output_path, "stock_selected.csv")
72     df_result.to_csv(output_file, index=False)

```

C.3 Sentiment Agent

Sentiment Agent - The Sentiment Agent uses a fine-tuned Llama-3.1-8B-Instruct model with LoRA adapters to analyze news sentiment and predict stock returns based on recent news and market data.

Listing 4: Sentiment Agent (sentiment_agent.py)

```

1  """
2  Sentiment Agent - Uses FinGPT to analyze news and generate trading signals
3  Adapted from FinGPT_Forecaster
4  """
5
6  import os
7  import re
8  import torch
9  from transformers import AutoTokenizer, AutoModelForCausalLM
10 from peft import PeftModel
11 from news_fetcher.prompt_builder import PromptBuilder
12 from news_fetcher.data_fetcher import NewsDataFetcher
13
14
15 class SentimentAgent:
16     """
17     Sentiment analysis agent using FinGPT model
18     """
19
20     def __init__(self, model_path=None, tokenizer_path=None, hf_token=None):
21         """
22         Initialize sentiment agent
23
24         Args:
25             model_path: Path to base model (default: 'meta-llama/Llama-2-7b-chat-hf')
26             tokenizer_path: Path to tokenizer (default: same as model_path)
27             hf_token: HuggingFace token (if None, uses HF_TOKEN env var)
28         """
29         self.hf_token = hf_token or os.environ.get("HF_TOKEN")
30         if not self.hf_token:
31             raise ValueError("HuggingFace token is required. Set HF_TOKEN environment variable.")
32
33         # Default model paths
34         base_model_path = model_path or 'meta-llama/Llama-2-7b-chat-hf'

```

```

35     lora_model_path = 'FinGPT/fingpt-forecaster_dow30_llama2-7b_lora'
36     tokenizer_path = tokenizer_path or base_model_path
37
38     # Load base model
39     self.base_model = AutoModelForCausalLM.from_pretrained(
40         base_model_path,
41         token=self.hf_token,
42         trust_remote_code=True,
43         device_map="auto",
44         dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
45     )
46
47     # Load LoRA adapter
48     try:
49         self.model = PeftModel.from_pretrained(self.base_model, lora_model_path)
50     except:
51         self.model = self.base_model
52
53     self.model = self.model.eval()
54
55     # Load tokenizer
56     self.tokenizer = AutoTokenizer.from_pretrained(tokenizer_path, token=self.hf_token)
57
58     # Initialize prompt builder
59     self.prompt_builder = PromptBuilder()
60     self.data_fetcher = NewsDataFetcher()
61
62     # Prompt format constants
63     self.B_INST, self.E_INST = "[INST]", "[/INST]"
64     self.B_SYS, self.E_SYS = "<<SYS>>\n", "\n<</SYS>>\n\n"
65
66     self.SYSTEM_PROMPT = (
67         "You are a seasoned stock market analyst. Your task is to list the positive "
68         "developments and potential concerns for companies based on relevant news and "
69         "basic financials from the past weeks, then provide an analysis and prediction "
70         "for the companies' stock price movement for the upcoming week."
71     )
72
73     def analyze(self, news_data, symbol, date, n_weeks=3):
74         """
75         Analyze news sentiment and generate trading signal
76
77         Args:
78             news_data: DataFrame with news data (optional, if None will fetch)
79             symbol: Stock ticker symbol
80             date: Trading date string
81             n_weeks: Number of weeks to look back
82
83         Returns:
84             dict: {
85                 'signal': 1 (buy), -1 (sell), 0 (hold),
86                 'confidence': 0.0-1.0,
87                 'predicted_return': float,
88                 'raw_output': str
89             }
90         """
91         # Fetch data if not provided
92         if news_data is None:
93             news_data = self.data_fetcher.fetch_all_data(symbol, date, n_weeks)
94
95         # Build prompt

```

```

96     info, prompt = self.prompt_builder.get_all_prompts_online(
97         symbol, news_data, date, with_basics=True
98     )
99
100     # Format prompt for Llama
101     formatted_prompt = (
102         self.B_INST + self.B_SYS + self.SYSTEM_PROMPT + self.E_SYS +
103         prompt + self.E_INST
104     )
105
106     # Generate prediction
107     inputs = self.tokenizer(formatted_prompt, return_tensors='pt')
108     inputs = {key: value.to(self.model.device) for key, value in inputs.items()}
109
110     with torch.no_grad():
111         res = self.model.generate(
112             **inputs,
113             max_length=4096,
114             do_sample=True,
115             temperature=0.7,
116             top_p=0.9
117         )
118
119     output = self.tokenizer.decode(res[0], skip_special_tokens=True)
120
121     # Parse output
122     result = self._parse_output(output, symbol)
123     result['raw_output'] = output
124
125     return result
126
127 def _parse_output(self, output, symbol):
128     """
129     Parse FinGPT output to extract signal
130
131     Args:
132         output: Raw model output
133         symbol: Stock ticker symbol (for context)
134
135     Returns:
136         dict with signal, confidence, etc.
137     """
138     # Extract prediction - support both single value and range formats
139     prediction_match = re.search(
140         r'Prediction:\s*(?:up|down)\s*(?:by)?\s*([\d.]+)(?:-([\d.]+))?%',
141         output,
142         re.IGNORECASE
143     )
144
145     predicted_return = 0.0
146     if prediction_match:
147         if prediction_match.group(2): # Range format
148             value1 = float(prediction_match.group(1))
149             value2 = float(prediction_match.group(2))
150             value = (value1 + value2) / 2
151         else: # Single value format
152             value = float(prediction_match.group(1))
153
154         # Check if up or down
155         prediction_text = prediction_match.group(0).lower()
156         if 'up' in prediction_text or 'increase' in output.lower():

```

```

157         predicted_return = value / 100
158     elif 'down' in prediction_text or 'decrease' in output.lower():
159         predicted_return = -value / 100
160     else:
161         predicted_return = value / 100
162
163     # Generate signal based on prediction
164     if predicted_return > 0.02: # Predict > 2% increase
165         signal = 1
166         confidence = min(abs(predicted_return) * 10, 1.0)
167     elif predicted_return < -0.02: # Predict > 2% decrease
168         signal = -1
169         confidence = min(abs(predicted_return) * 10, 1.0)
170     else:
171         signal = 0
172         confidence = 0.5
173
174     return {
175         'signal': signal,
176         'confidence': confidence,
177         'predicted_return': predicted_return
178     }

```

C.4 Technical Agent

Technical Agent - The Technical Agent uses technical indicators (RSI, MACD, CCI, ADX) to predict returns and applies mean-variance optimization to calculate optimal portfolio weights.

Listing 5: Technical Agent Optimized (technical_agent_optimized.py)

```

1  """
2  Technical Agent Optimized - Uses Mean-Variance Optimization
3  Based on FinRL portfolio optimization approach
4  """
5
6  import pandas as pd
7  import numpy as np
8  from typing import Dict, List, Optional
9
10
11  class TechnicalAgentOptimized:
12      """
13      Improved Technical Agent that uses mean-variance optimization
14      to calculate portfolio weights based on technical indicators
15      """
16
17      def __init__(self, indicators_config=None, use_optimization=True):
18          """
19          Initialize optimized technical agent
20
21          Args:
22              indicators_config: dict with thresholds for each indicator
23              use_optimization: whether to use mean-variance optimization
24          """
25          self.config = indicators_config or self._default_config()
26          self.use_optimization = use_optimization
27
28          # Try to import PyPortfolioOpt
29          try:
30              from pypfport.efficient_frontier import EfficientFrontier
31              from pypfport import risk_models, objective_functions

```

```

32         self.optimization_available = True
33     except ImportError:
34         print("Warning: PyPortfolioOpt not available. Falling back to simple method.")
35         self.optimization_available = False
36         self.use_optimization = False
37
38     def _default_config(self):
39         """Default configuration for technical indicators"""
40         return {
41             'rsi_oversold': 30,
42             'rsi_overbought': 70,
43             'cci_oversold': -100,
44             'cci_overbought': 100,
45             'adx_trend_threshold': 25,
46             'macd_bullish_threshold': 0,
47             'weight_bounds': (0, 0.05), # Max 5% per stock
48             'min_history_days': 252 # 1 year of data
49         }
50
51     def analyze_portfolio(self, tech_data_dict: Dict, historical_returns: pd.DataFrame) ->
52         Dict:
53         """
54         Analyze multiple stocks using technical indicators and optimize portfolio weights
55
56         Args:
57             tech_data_dict: dict of {gvkey: {rsi, macd, cci, adx, ...}}
58             historical_returns: DataFrame with historical returns
59                 - columns: stock identifiers (gvkey or tic)
60                 - index: dates
61                 - values: daily returns
62
63         Returns:
64             dict: {
65                 'weights': {gvkey: weight},
66                 'expected_returns': {gvkey: expected_return},
67                 'confidence': float,
68                 'sharpe_ratio': float,
69                 'method': str,
70                 'signal': float # Overall portfolio signal
71             }
72         """
73         if not self.use_optimization or not self.optimization_available:
74             return self._analyze_simple(tech_data_dict)
75
76         # Check if we have enough data
77         if len(historical_returns) < self.config['min_history_days']:
78             return self._analyze_simple(tech_data_dict)
79
80         # 1. Predict expected returns from technical indicators
81         predicted_returns = {}
82         for gvkey, tech_data in tech_data_dict.items():
83             predicted_returns[gvkey] = self._predict_return_from_indicators(tech_data)
84
85         # 2. Filter stocks that exist in both predicted_returns and historical_returns
86         common_stocks = set(predicted_returns.keys()) & set(historical_returns.columns)
87         if len(common_stocks) == 0:
88             return self._analyze_simple(tech_data_dict)
89
90         # 3. Prepare data for optimization
91         mu = pd.Series({s: predicted_returns[s] for s in common_stocks})
92         S = historical_returns[list(common_stocks)].cov()

```

```

92
93     # 4. Mean-variance optimization
94     try:
95         from pypfopt.efficient_frontier import EfficientFrontier
96         from pypfopt import risk_models, objective_functions
97
98         ef = EfficientFrontier(mu, S, weight_bounds=self.config['weight_bounds'])
99         raw_weights = ef.nonconvex_objective(
100             objective_functions.sharpe_ratio,
101             objective_args=(ef.expected_returns, ef.cov_matrix),
102             weights_sum_to_one=True
103         )
104         cleaned_weights = ef.clean_weights()
105
106         # Re-normalize weights after cleaning
107         total_weight = sum(cleaned_weights.values())
108         if total_weight > 0:
109             cleaned_weights = {k: v / total_weight for k, v in cleaned_weights.items()}
110
111         # 5. Calculate portfolio performance metrics
112         performance = ef.portfolio_performance(verbose=False)
113         expected_return = performance[0]
114         volatility = performance[1]
115         sharpe_ratio = performance[2]
116
117         # 6. Calculate overall signal
118         overall_signal = sum(cleaned_weights.get(s, 0) * predicted_returns.get(s, 0)
119                             for s in common_stocks)
120
121         # 7. Confidence based on Sharpe ratio and signal strength
122         confidence = min(abs(sharpe_ratio) / 2.0, 1.0) if sharpe_ratio is not None else
123             0.5
124         if abs(overall_signal) > 0.1:
125             confidence = min(confidence * 1.2, 1.0)
126
127         return {
128             'weights': cleaned_weights,
129             'expected_returns': predicted_returns,
130             'confidence': confidence,
131             'sharpe_ratio': sharpe_ratio,
132             'expected_return': expected_return,
133             'volatility': volatility,
134             'method': 'mean_variance_optimization',
135             'signal': np.tanh(overall_signal * 10)
136         }
137
138     except Exception as e:
139         print(f"Error in optimization: {e}")
140         return self._analyze_simple(tech_data_dict)
141
142 def _predict_return_from_indicators(self, tech_data: Dict) -> float:
143     """
144     Predict expected return from technical indicators
145
146     Args:
147         tech_data: dict with technical indicators
148
149     Returns:
150         float: predicted return
151     """
152     rsi = tech_data.get('rsi', 50)

```

```

152     macd = tech_data.get('macd', 0)
153     macd_signal = tech_data.get('macd_signal', 0)
154     cci = tech_data.get('cci', 0)
155     adx = tech_data.get('adx', 0)
156
157     # Handle NaN values
158     rsi = 50 if pd.isna(rsi) else rsi
159     macd = 0 if pd.isna(macd) else macd
160     macd_signal = 0 if pd.isna(macd_signal) else macd_signal
161     cci = 0 if pd.isna(cci) else cci
162     adx = 0 if pd.isna(adx) else adx
163
164     # RSI: Low RSI (< 30) -> oversold -> positive expected return
165     rsi_score = (50 - rsi) / 50 # Normalize to [-1, 1]
166
167     # MACD: MACD > Signal -> bullish -> positive expected return
168     macd_diff = macd - macd_signal
169     macd_score = np.tanh(macd_diff * 10) # Normalize
170
171     # CCI: Low CCI (< -100) -> oversold -> positive expected return
172     cci_score = -cci / 200 # Normalize to approximately [-1, 1]
173
174     # ADX: High ADX (> 25) -> strong trend -> increase confidence
175     adx_factor = 1.0 + (adx / 50) if adx > self.config['adx_trend_threshold'] else 1.0
176
177     # Combine indicators (weighted average)
178     predicted_return = (
179         rsi_score * 0.4 +
180         macd_score * 0.3 +
181         cci_score * 0.3
182     ) * 0.1 * adx_factor # Scale to reasonable return range
183
184     return predicted_return
185
186 def _analyze_simple(self, tech_data_dict: Dict) -> Dict:
187     """Fallback method: simple analysis without optimization"""
188     n_stocks = len(tech_data_dict)
189     if n_stocks == 0:
190         return {
191             'weights': {},
192             'confidence': 0.0,
193             'signal': 0.0,
194             'method': 'simple_equal_weight'
195         }
196
197     equal_weight = 1.0 / n_stocks
198     weights = {gvkey: equal_weight for gvkey in tech_data_dict.keys()}
199
200     return {
201         'weights': weights,
202         'confidence': 0.5,
203         'signal': 0.0,
204         'method': 'simple_equal_weight'
205     }

```

C.5 Risk Manager

Risk Manager - The Risk Manager calculates volatility-adjusted and correlation-adjusted position limits for each stock to control portfolio risk.

Listing 6: Risk Manager (risk_manager.py)

```

1 from langchain_core.messages import HumanMessage
2 from src.graph.state import AgentState, show_agent_reasoning
3 from src.utils.progress import progress
4 from src.tools.api import get_prices, prices_to_df
5 import json
6 import numpy as np
7 import pandas as pd
8 from src.utils.api_key import get_api_key_from_state
9
10 ##### Risk Management Agent #####
11 def risk_management_agent(state: AgentState, agent_id: str = "risk_management_agent"):
12     """Controls position sizing based on volatility-adjusted risk factors for multiple
13     tickers."""
14     portfolio = state["data"]["portfolio"]
15     data = state["data"]
16     tickers = data["tickers"]
17     api_key = get_api_key_from_state(state, "FINANCIAL_DATASETS_API_KEY")
18
19     # Initialize risk analysis for each ticker
20     risk_analysis = {}
21     current_prices = {}
22     volatility_data = {}
23     returns_by_ticker: dict[str, pd.Series] = {}
24
25     # First, fetch prices and calculate volatility for all relevant tickers
26     all_tickers = set(tickers) | set(portfolio.get("positions", {}).keys())
27
28     for ticker in all_tickers:
29         prices = get_prices(
30             ticker=ticker,
31             start_date=data["start_date"],
32             end_date=data["end_date"],
33             api_key=api_key,
34         )
35
36         if not prices:
37             volatility_data[ticker] = {
38                 "daily_volatility": 0.05,
39                 "annualized_volatility": 0.05 * np.sqrt(252),
40                 "volatility_percentile": 100,
41                 "data_points": 0
42             }
43             continue
44
45         prices_df = prices_to_df(prices)
46
47         if not prices_df.empty and len(prices_df) > 1:
48             current_price = prices_df["close"].iloc[-1]
49             current_prices[ticker] = current_price
50
51             # Calculate volatility metrics
52             volatility_metrics = calculate_volatility_metrics(prices_df)
53             volatility_data[ticker] = volatility_metrics
54
55             # Store returns for correlation analysis
56             daily_returns = prices_df["close"].pct_change().dropna()
57             if len(daily_returns) > 0:
58                 returns_by_ticker[ticker] = daily_returns
59
60     # Build returns DataFrame aligned across tickers for correlation analysis

```



```

60 correlation_matrix = None
61 if len(returns_by_ticker) >= 2:
62     try:
63         returns_df = pd.DataFrame(returns_by_ticker).dropna(how="any")
64         if returns_df.shape[1] >= 2 and returns_df.shape[0] >= 5:
65             correlation_matrix = returns_df.corr()
66     except Exception:
67         correlation_matrix = None
68
69 # Determine which tickers currently have exposure
70 active_positions = {
71     t for t, pos in portfolio.get("positions", {}).items()
72     if abs(pos.get("long", 0) - pos.get("short", 0)) > 0
73 }
74
75 # Calculate total portfolio value
76 total_portfolio_value = portfolio.get("cash", 0.0)
77 for ticker, position in portfolio.get("positions", {}).items():
78     if ticker in current_prices:
79         total_portfolio_value += position.get("long", 0) * current_prices[ticker]
80         total_portfolio_value -= position.get("short", 0) * current_prices[ticker]
81
82 # Calculate volatility- and correlation-adjusted risk limits for each ticker
83 for ticker in tickers:
84     if ticker not in current_prices or current_prices[ticker] <= 0:
85         risk_analysis[ticker] = {
86             "remaining_position_limit": 0.0,
87             "current_price": 0.0,
88             "reasoning": {"error": "Missing price data for risk calculation"}
89         }
90         continue
91
92     current_price = current_prices[ticker]
93     vol_data = volatility_data.get(ticker, {})
94
95     # Calculate current market value of this position
96     position = portfolio.get("positions", {}).get(ticker, {})
97     long_value = position.get("long", 0) * current_price
98     short_value = position.get("short", 0) * current_price
99     current_position_value = abs(long_value - short_value)
100
101     # Volatility-adjusted limit pct
102     vol_adjusted_limit_pct = calculate_volatility_adjusted_limit(
103         vol_data.get("annualized_volatility", 0.25)
104     )
105
106     # Correlation adjustment
107     corr_metrics = {
108         "avg_correlation_with_active": None,
109         "max_correlation_with_active": None,
110         "top_correlated_tickers": [],
111     }
112     corr_multiplier = 1.0
113     if correlation_matrix is not None and ticker in correlation_matrix.columns:
114         comparable = [t for t in active_positions if t in correlation_matrix.columns and
115             t != ticker]
116         if not comparable:
117             comparable = [t for t in correlation_matrix.columns if t != ticker]
118         if comparable:
119             series = correlation_matrix.loc[ticker, comparable]
120             series = series.dropna()

```

```

120         if len(series) > 0:
121             avg_corr = float(series.mean())
122             max_corr = float(series.max())
123             corr_metrics["avg_correlation_with_active"] = avg_corr
124             corr_metrics["max_correlation_with_active"] = max_corr
125             top_corr = series.sort_values(ascending=False).head(3)
126             corr_metrics["top_correlated_tickers"] = [
127                 {"ticker": idx, "correlation": float(val)} for idx, val in
128                 top_corr.items()
129             ]
130             corr_multiplier = calculate_correlation_multiplier(avg_corr)
131
132             # Combine volatility and correlation adjustments
133             combined_limit_pct = vol_adjusted_limit_pct * corr_multiplier
134             position_limit = total_portfolio_value * combined_limit_pct
135
136             # Calculate remaining limit for this position
137             remaining_position_limit = position_limit - current_position_value
138
139             # Ensure we don't exceed available cash
140             max_position_size = min(remaining_position_limit, portfolio.get("cash", 0))
141
142             risk_analysis[ticker] = {
143                 "remaining_position_limit": float(max_position_size),
144                 "current_price": float(current_price),
145                 "volatility_metrics": {
146                     "daily_volatility": float(vol_data.get("daily_volatility", 0.05)),
147                     "annualized_volatility": float(vol_data.get("annualized_volatility", 0.25)),
148                     "volatility_percentile": float(vol_data.get("volatility_percentile", 100)),
149                     "data_points": int(vol_data.get("data_points", 0))
150                 },
151                 "correlation_metrics": corr_metrics,
152                 "reasoning": {
153                     "portfolio_value": float(total_portfolio_value),
154                     "current_position_value": float(current_position_value),
155                     "base_position_limit_pct": float(vol_adjusted_limit_pct),
156                     "correlation_multiplier": float(corr_multiplier),
157                     "combined_position_limit_pct": float(combined_limit_pct),
158                     "position_limit": float(position_limit),
159                     "remaining_limit": float(remaining_position_limit),
160                     "available_cash": float(portfolio.get("cash", 0)),
161                 },
162             }
163
164             # Add the signal to the analyst_signals list
165             state["data"]["analyst_signals"][agent_id] = risk_analysis
166
167         return {
168             "messages": state["messages"] + [message],
169             "data": data,
170         }
171
172     def calculate_volatility_metrics(prices_df: pd.DataFrame, lookback_days: int = 60) -> dict:
173         """Calculate comprehensive volatility metrics from price data."""
174         if len(prices_df) < 2:
175             return {
176                 "daily_volatility": 0.05,
177                 "annualized_volatility": 0.05 * np.sqrt(252),
178                 "volatility_percentile": 100,
179                 "data_points": len(prices_df)

```

```

180     }
181
182     # Calculate daily returns
183     daily_returns = prices_df["close"].pct_change().dropna()
184
185     if len(daily_returns) < 2:
186         return {
187             "daily_volatility": 0.05,
188             "annualized_volatility": 0.05 * np.sqrt(252),
189             "volatility_percentile": 100,
190             "data_points": len(daily_returns)
191         }
192
193     # Use the most recent lookback_days for volatility calculation
194     recent_returns = daily_returns.tail(min(lookback_days, len(daily_returns)))
195
196     # Calculate volatility metrics
197     daily_vol = recent_returns.std()
198     annualized_vol = daily_vol * np.sqrt(252) # Annualize assuming 252 trading days
199
200     # Calculate percentile rank of recent volatility vs historical volatility
201     if len(daily_returns) >= 30:
202         rolling_vol = daily_returns.rolling(window=30).std().dropna()
203         if len(rolling_vol) > 0:
204             current_vol_percentile = (rolling_vol <= daily_vol).mean() * 100
205         else:
206             current_vol_percentile = 50
207     else:
208         current_vol_percentile = 50
209
210     return {
211         "daily_volatility": float(daily_vol) if not np.isnan(daily_vol) else 0.025,
212         "annualized_volatility": float(annualized_vol) if not np.isnan(annualized_vol) else
213         0.25,
214         "volatility_percentile": float(current_vol_percentile) if not
215         np.isnan(current_vol_percentile) else 50.0,
216         "data_points": len(recent_returns)
217     }
218
219 def calculate_volatility_adjusted_limit(annualized_volatility: float) -> float:
220     """
221     Calculate position limit as percentage of portfolio based on volatility.
222
223     Logic:
224     - Low volatility (<15%): Up to 25% allocation
225     - Medium volatility (15-30%): 15-20% allocation
226     - High volatility (>30%): 10-15% allocation
227     - Very high volatility (>50%): Max 10% allocation
228     """
229     base_limit = 0.20 # 20% baseline
230
231     if annualized_volatility < 0.15: # Low volatility
232         vol_multiplier = 1.25 # Up to 25%
233     elif annualized_volatility < 0.30: # Medium volatility
234         vol_multiplier = 1.0 - (annualized_volatility - 0.15) * 0.5 # 20% -> 12.5%
235     elif annualized_volatility < 0.50: # High volatility
236         vol_multiplier = 0.75 - (annualized_volatility - 0.30) * 0.5 # 15% -> 5%
237     else: # Very high volatility (>50%)
238         vol_multiplier = 0.50 # Max 10%

```

```

239     # Apply bounds to ensure reasonable limits
240     vol_multiplier = max(0.25, min(1.25, vol_multiplier)) # 5% to 25% range
241
242     return base_limit * vol_multiplier
243
244
245 def calculate_correlation_multiplier(avg_correlation: float) -> float:
246     """Map average correlation to an adjustment multiplier.
247     - Very high correlation (>= 0.8): reduce limit sharply (0.7x)
248     - High correlation (0.6-0.8): reduce (0.85x)
249     - Moderate correlation (0.4-0.6): neutral (1.0x)
250     - Low correlation (0.2-0.4): slight increase (1.05x)
251     - Very low correlation (< 0.2): increase (1.10x)
252     """
253     if avg_correlation >= 0.80:
254         return 0.70
255     if avg_correlation >= 0.60:
256         return 0.85
257     if avg_correlation >= 0.40:
258         return 1.00
259     if avg_correlation >= 0.20:
260         return 1.05
261     return 1.10

```

C.6 Integration Flow

The complete trading pipeline follows this sequence:

1. **Stock Selection:** ML models predict forward returns per sector, select top 25%
2. **Sentiment Analysis:** FinGPT analyzes news and generates predicted returns
3. **Technical Analysis:** Technical indicators + mean-variance optimization generate weights
4. **Risk Management:** Calculate position limits based on volatility and correlation
5. **Portfolio Manager:** Filter by sentiment, apply technical weights, generate final portfolio

References

- [1] Hongyang (Bruce) Yang, Xiao-Yang Liu, and Qingwei Wu. A practical machine learning approach for dynamic stock recommendation. SSRN working paper, 2025. Department of Statistics and Department of Electrical Engineering, Columbia University.